



The
University of
Faisalabad

PROJECT REPORT

Weather Forecasting Using Data Mining and Predictive Analytics Models

Deep Learning Based Weather Prediction System

Submitted By

Roll Number	Name	Program
2023-BS-AI-153	Mohammad Saud	BS Artificial Intelligence
2023-BS-AI-133	Nouman Khalid	BS Artificial Intelligence
2023-BS-AI-129	Daniyal Hussain	BS Artificial Intelligence

Session: 2023 – 2027 | Semester: 6th | Session: 2026
June 2026

1. Introduction

Weather forecasting is one of the most critical applications of data science and machine learning, affecting agriculture, transportation, disaster management, and everyday human activity. Traditional numerical weather prediction models require immense computational resources and expert meteorological knowledge. With the rapid advancement of deep learning, it is now possible to build accurate, data-driven weather prediction systems that learn patterns directly from historical data.

This project presents a deep learning-based weather prediction system developed using the Seattle Weather dataset. The system incorporates two distinct predictive models: an Artificial Neural Network (ANN) for weather type classification, and a Long Short-Term Memory (LSTM) network for next-day temperature forecasting. The application is deployed as an interactive web interface using Streamlit, providing a user-friendly experience for both prediction tasks.

2. Objectives

The primary objectives of this project are:

- To develop a weather type classification model using an Artificial Neural Network (ANN) that predicts weather conditions such as rain, sun, fog, drizzle, and snow based on atmospheric features.
- To build an LSTM-based time-series model capable of forecasting next-day maximum and minimum temperatures using a 7-day historical sequence.
- To apply data preprocessing techniques including feature scaling with MinMaxScaler and label encoding for categorical targets.
- To deploy both models as an interactive, multi-page Streamlit web application accessible to end users.
- To evaluate model performance using standard metrics: classification accuracy for the ANN and Mean Absolute Error (MAE) for the LSTM forecaster.
- To explore the practical application of deep learning in meteorological prediction using real-world weather data.

3. Dataset Description

The project uses the Seattle Weather dataset ([seattle-weather.csv](#)), a publicly available tabular dataset containing daily weather observations from Seattle, Washington. The dataset captures a variety of atmospheric measurements that serve as inputs for both predictive models.

3.1 Dataset Features

Feature	Data Type	Description
date	DateTime	Date of the weather observation

Feature	Data Type	Description
precipitation	Float	Amount of precipitation in mm
temp_max	Float	Maximum temperature recorded for the day (°C)
temp_min	Float	Minimum temperature recorded for the day (°C)
wind	Float	Wind speed in km/h
weather	String	Target label: rain, sun, fog, drizzle, snow

The date column is converted to a datetime format during preprocessing but is not used as a direct model input. The four numerical features — precipitation, temp_max, temp_min, and wind — are used for classification, while temp_max and temp_min form the input sequence for the LSTM forecasting model.

4. System Architecture

The system is built on a two-model architecture trained and saved independently, then loaded at runtime by the Streamlit application. The high-level workflow is as follows:

4.1 Model Training Pipeline (model.py)

- Dataset is loaded and the date column is parsed as a datetime object.
- For classification: features are scaled using MinMaxScaler; labels are integer-encoded using LabelEncoder and one-hot encoded for ANN training.
- For forecasting: temp_max and temp_min are normalized; sliding windows of length 7 are constructed to form supervised time-series sequences.
- Models are trained with an 80/20 train-test split and a 20% validation split during training.
- All trained artifacts — models, scalers, and encoder — are persisted to the models/ directory.

4.2 Application Layer (app.py)

- Loads all saved models and preprocessing artifacts at startup.
- Provides three navigable pages via a sidebar: Home, Weather Prediction, and Temperature Forecasting.
- The Home page displays dataset statistics and visualizations of weather distribution and temperature trends.
- The Weather Prediction page accepts four numerical inputs and returns the predicted weather class and confidence score.
- The Temperature Forecasting page accepts 7 days of temperature readings and returns the predicted next-day max and min temperatures.

5. Machine Learning Models

5.1 Weather Classification – Artificial Neural Network (ANN)

The classification model is a fully connected feedforward neural network trained to categorize daily weather into five classes: rain, sun, fog, drizzle, and snow. The network architecture is as follows:

Layer	Type	Units	Activation
Input Layer	Dense	64	ReLU
Dropout	Dropout	—	Rate 0.3
Hidden Layer	Dense	32	ReLU
Dropout	Dropout	—	Rate 0.2
Output Layer	Dense	5 classes	Softmax

Training configuration:

- Optimizer: Adam
- Loss Function: Categorical Cross-Entropy
- Epochs: 30 | Batch Size: 16 | Validation Split: 20%
- Evaluation Metric: Classification Accuracy

5.2 Temperature Forecasting – LSTM Network

The forecasting model is a stacked LSTM network designed for time-series regression. It takes a sequence of 7 consecutive days' maximum and minimum temperatures and predicts the temperature values for the following day.

Layer	Type	Units	Notes
LSTM Layer 1	LSTM	64	return_sequences=True
Dropout	Dropout	—	Rate 0.2
LSTM Layer 2	LSTM	32	—
Dense	Dense	16	ReLU
Output	Dense	2	Linear (max, min temp)

Training configuration:

- Optimizer: Adam
- Loss Function: Mean Squared Error (MSE)
- Epochs: 30 | Batch Size: 16 | Validation Split: 20%

- Evaluation Metric: Mean Absolute Error (MAE)

6. Implementation Details

6.1 Technology Stack

Component	Technology / Library
Programming Language	Python 3.x
Deep Learning Framework	TensorFlow / Keras
Data Processing	Pandas, NumPy
Preprocessing	Scikit-learn (MinMaxScaler, LabelEncoder)
Model Persistence	Joblib (.pkl), Keras native format (.keras)
Web Application	Streamlit
Visualization	Matplotlib
Dataset	Seattle Weather CSV (seattle-weather.csv)

6.2 File Structure

- `model.py` — Training script; builds, trains, evaluates, and saves all models and preprocessors.
- `app.py` — Streamlit application; loads models and provides the interactive user interface.
- `seattle-weather.csv` — Raw dataset used for training and display.
- `models/weather_classifier.keras` — Saved ANN classification model.
- `models/temperature_forecaster.keras` — Saved LSTM forecasting model.
- `models/scaler.pkl` — MinMaxScaler fitted on the four classification features.
- `models/temp_scaler.pkl` — MinMaxScaler fitted on temperature data for the LSTM model.
- `models/label_encoder.pkl` — LabelEncoder mapping weather class names to integers.

6.3 Preprocessing Pipeline

Both models rely on MinMaxScaler to normalize input features to the $[0, 1]$ range before training, which is critical for neural network convergence. The scalers fitted on training data are saved and reloaded at inference time to ensure consistent transformations. For classification, the target labels are integer-encoded with LabelEncoder and then converted to one-hot vectors using `_categorical`. For the LSTM model, the `inverse_transform` step restores predicted normalized values back to the original Celsius scale before display.

7. Application Features

7.1 Home Page

The Home page provides an exploratory overview of the dataset. It renders the first few rows of the CSV file, a full statistical summary (count, mean, std, min/max, quartiles) via `df.describe()`, a bar chart showing the distribution of each weather type, and a line chart comparing the temperature trends of maximum and minimum temperature over the entire dataset.

7.2 Weather Prediction Page

Users can input four values — precipitation, maximum temperature, minimum temperature, and wind speed — and click Predict Weather. The inputs are scaled using the saved scaler, passed through the ANN classifier, and the softmax output is decoded via the label encoder. The page displays the predicted weather type and a confidence percentage derived from the maximum class probability.

7.3 Temperature Forecasting Page

This page collects 7 days of historical temperature data using a dynamic two-column input grid (maximum and minimum temperature per day). On clicking Forecast Temperature, the sequence is normalized, reshaped to the LSTM input format (1, 7, 2), and passed through the forecasting model. The inverse-transformed predictions are displayed as the expected next-day maximum and minimum temperatures in degrees Celsius.

8. Model Evaluation

Both models are evaluated on a held-out test set comprising 20% of the dataset, ensuring that performance metrics reflect generalization rather than memorization.

Model	Task	Metric	Target
ANN Classifier	Weather Classification	Accuracy	Higher is better
LSTM Forecaster	Temperature Forecasting	Mean Absolute Error	Lower is better

The ANN classifier is evaluated using `sklearn's accuracy_score` comparing `argmax`-decoded predictions against integer-encoded ground truth labels. The LSTM forecaster is evaluated using `mean_absolute_error` after inverse-transforming both predictions and ground truth back to the original Celsius scale, ensuring interpretable error magnitudes.

9. Conclusion

This project successfully demonstrates the application of deep learning techniques to real-world weather prediction tasks. By combining an ANN classifier and an LSTM time-series forecaster within a single Streamlit application, the system provides both categorization of weather types and quantitative temperature forecasting from a user-friendly interface.

The use of MinMaxScaler normalization, dropout regularization, and appropriate loss functions for each task reflects sound deep learning practices. The modular design, separating training (`model.py`) from inference (`app.py`), ensures the system is maintainable and easy to extend.

Future work could include expanding the dataset to multiple cities, incorporating additional meteorological features such as humidity and atmospheric pressure, experimenting with Transformer architectures for the forecasting task, and deploying the application to a cloud platform for broader accessibility.

10. References

- [1] Abadi, M. et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. USENIX OSDI.
- [2] Chollet, F. (2018). Deep Learning with Python. Manning Publications.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
- [5] Streamlit Inc. (2023). Streamlit Documentation. <https://docs.streamlit.io>
- [6] Seattle Weather Dataset. Kaggle. <https://www.kaggle.com/datasets/ananthr1/weather-prediction>